

Introduction à la création de sites web dynamiques en PHP / MySQL

2ème partie: MySQL

Quentin Brossard, D&FI, juin/juillet 2003

<http://home.tiscalinet.ch/qbrossard>

Plan du cours (3)

- (My)SQL (4h)
 - Introduction et mise en oeuvre de MySQL/PHPMyAdmin
 - Introduction aux schémas Entités/Associations
 - Passage schéma E/A vers base de données
 - Introduction au langage SQL
 - Interrogation, Insertion, mise-à-jour et suppression des données
 - Spécificités SQL de MySQL

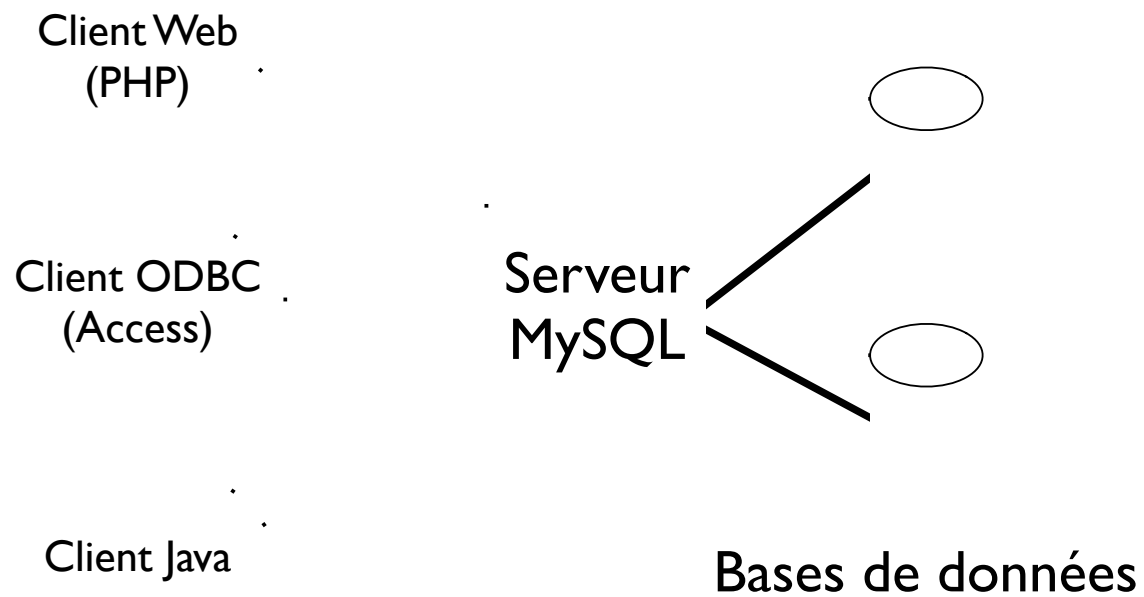
(My)SQL

- Créée en 1995
- Open source mais développée et supportée par une société commerciale suédoise (<http://www.mysql.com>)
- LE standard chez les fournisseurs de hosting web
- environ 4 millions d'installations

(My)SQL

- Multi-plateformes:
 - UNIX/Linux, Windows, MacOS X
- Client/serveur
- Nombreuses librairies clientes:
 - Java, C, C++
 - PHP, ASP, ODBC

(My)SQL



(My)SQL

- Limitations:
 - Pas (encore) transactionnelle
 - Manque de certaines fonctions high end (clustering, partitionnement)
 - Limitations dans l'implémentation SQL
 - Pas de clés étrangères!

(My)SQL

- Présentation des outils:
 - Serveur MySQL (bases de données, configuration)
 - client mysql (ligne de commandes)
 - PHPMyAdmin (www)
 - MySQL Control Center

(My)SQL:

Schémas Entités/Associations

- Permettent une modélisation conceptuelle de la base de données
- Vision Top Down
- Méthode simple mais puissante pour modéliser une base de données relationnelle
- Composés d'*Entités* (les tables) et d'*Associations* (les liens entre ces tables)

(My)SQL:

Schémas Entités/Associations

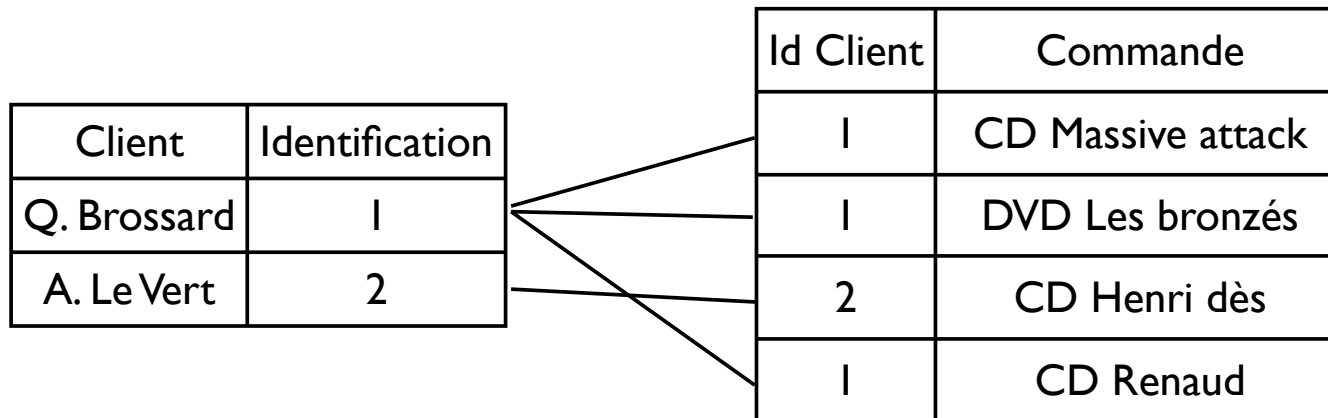
- Permettent la structuration et la *normalisation* des données
- Exemple: ***avant***

Client	Adresse	Commande1	Commande2	Commande3
Q.Brossard	St-Joseph, Carouge	CD Massive attack	DVD Les bronzés	
Q.Brossard	Lausanne	CD Renaud		
A. Le Vert	Genève	CD Henri Dès		

(My)SQL:

Schémas Entités/Associations

- Permettent la structuration et la *normalisation* des données
- Exemple: ***après***



(My)SQL:

Schémas Entités/Associations

- Les Entités
 - Composants identifiables de l'application
 - Nécessitent une *clé* pour être identifiés de manière unique. Le choix de cette clé est très important
 - Possèdent des propriétés appelées *attributs* dans les schémas E/A

(My)SQL:

Schémas Entités/Associations

- Les Entités
 - Ne modéliser que les entités utiles à l'application
 - Si la *clé* ne peut être facilement identifiée, créer un identifiant abstrait

Nom Entité
clé attribut 1 attribut 2 attribut 3

(My)SQL:

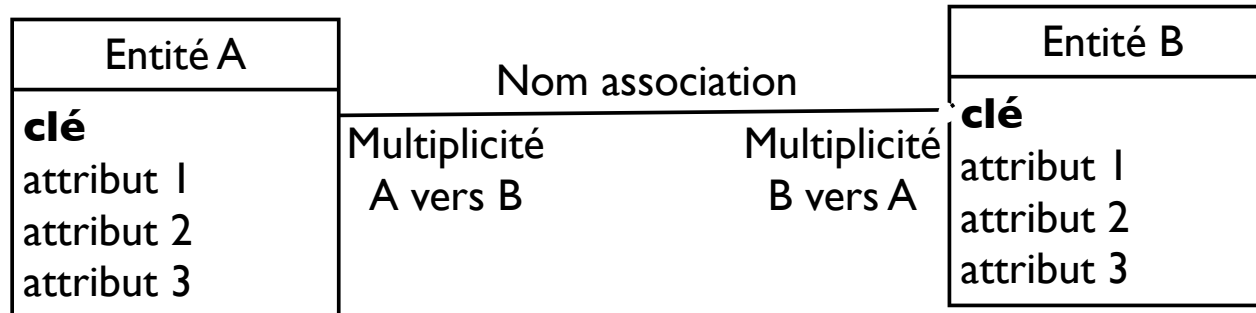
Schémas Entités/Associations

- Les associations
 - Permettent de modéliser les *relations* entre entités
 - peuvent être *multiples* ou *facultatives*
 - **multiples**: une entité A peut avoir plusieurs associations la liant à une autre entité B
 - **facultatives**: une association entre deux entités peut ne pas exister

(My)SQL:

Schémas Entités/Associations

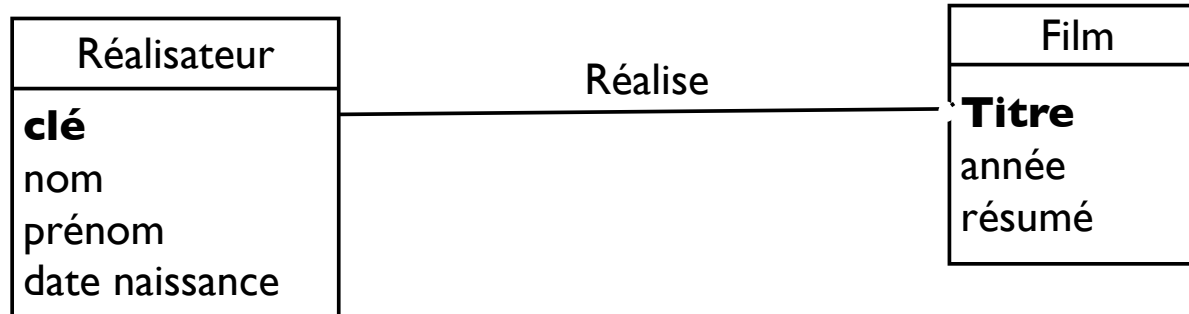
- La multiplicité est représentée par le nombre d'occurrences possibles:
 - un point à l'extrémité de l'association indique que cette entité peut être associée plusieurs fois à l'entité située à l'autre extrémité de l'association



(My)SQL:

Schémas Entités/Associations

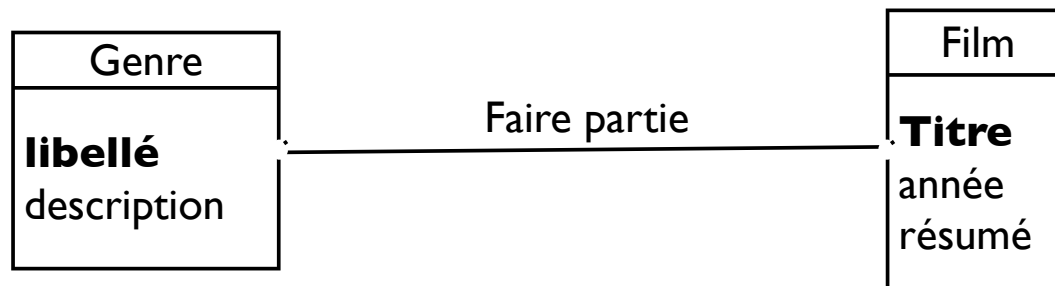
- Exemple: Films et réalisateurs
 - Un réalisateur peut avoir réalisé plusieurs films, mais un film peut avoir un seul réalisateur
 - Communément appelée association *père-fils*
 - Réalisateur possède une clé car le nom n'est pas un identifiant suffisant



(My)SQL:

Schémas Entités/Associations

- Exemple: Films et genre
 - Un genre peut comprendre plusieurs films et un film peut faire partie de plusieurs genres
 - Exemple: Les westerns spaghettis: Western ou comédie
 - Réponse: les deux



(My)SQL:

Schémas Entités/Associations

- Conseils:
 - Eviter les associations impliquant plus de deux entités
 - Représenter uniquement ce qu'on veut modéliser
 - Rester simple

(My)SQL:

Schémas Entités/Associations

- Passage au modèle logique de données:
 - le modèle logique de données représente dans la base de données
 - quelques règles simples permettent le passage du schéma E/A à la base de données

(My)SQL:

Schémas Entités/Associations

- Entités:
 - Chaque entité du schéma devient une table de même nom dans la base de données, cette table contient les mêmes attributs que l'entité
 - La clé doit permettre d'identifier chaque instance de l'entité de manière unique. Si aucun des attributs n'est approprié, on crée un champ clé. Un champ clé par entité est obligatoire

(My)SQL:

Schémas Entités/Associations

- Associations:
 - Un à plusieurs:
 - Après avoir créé les deux tables liées par l'association, on ajoute dans les attributs de la table fils (côté plusieurs) la clé de la table père
 - cet attribut est ce que l'on appelle une clé étrangère. Il permet d'assurer l'intégrité des données:
Que faire si le père est supprimé de la base de données?

(My)SQL:

Schémas Entités/Associations

- Associations:
 - Plusieurs à plusieurs:
 - Après avoir créé les deux tables liées par l'association, on crée une table contenant les clés des deux tables de l'association
 - La clé de cette nouvelle table est la paire de clés des deux tables (Ce type de clé multi-attributs est appelée clé composite)
 - Les éventuels attributs de l'association deviennent des attributs de la nouvelle table

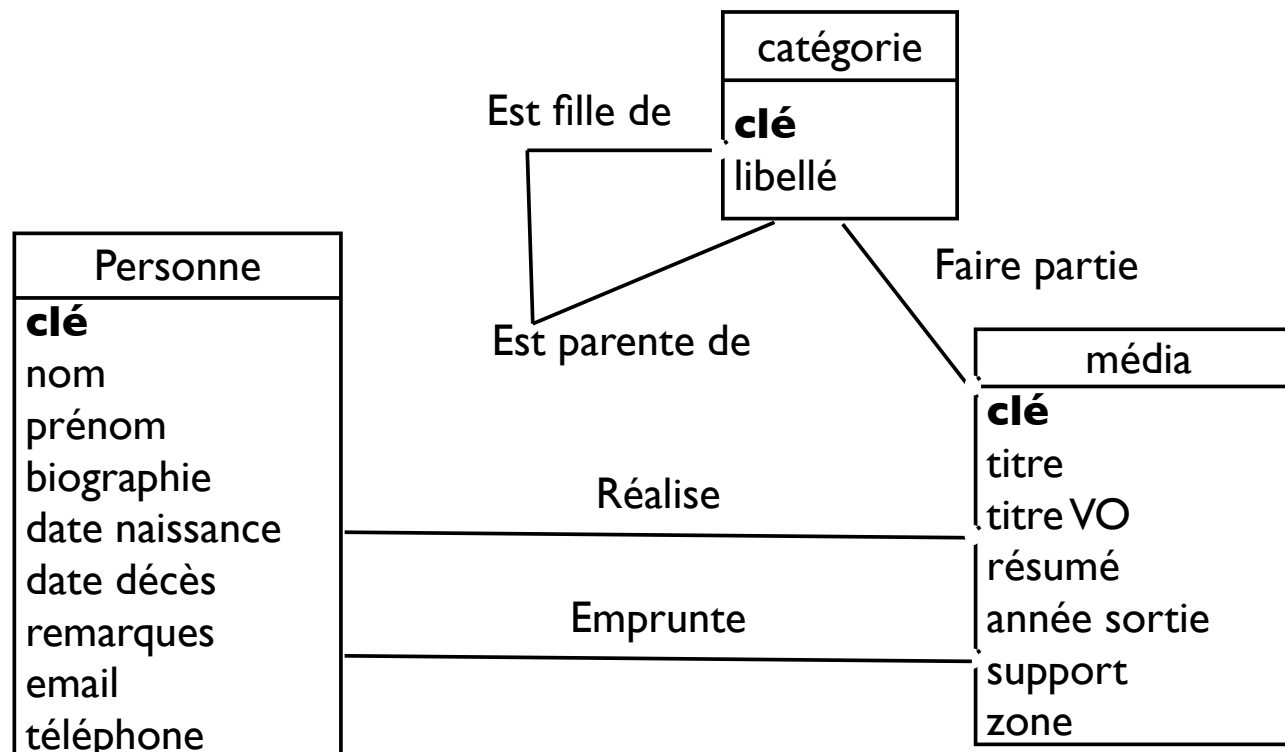
(My)SQL:

Schémas Entités/Associations

- Exemple: Gestion d'une petite médiathèque
 - Gestion de DVDs et de cassettes
 - Classement des médias par catégories
 - Gestion des réalisateurs, y compris dates naissance et commentaires
 - Gestion des prêts de DVD et cassettes aux amis...

(My)SQL:

Schémas Entités/Associations



(My)SQL:

Modèle logique de données

Personne
clé
nom
prénom
biographie
date naissance
date décès
remarques
email
téléphone

média
clé
titre
titre VO
résumé
année sortie
support
zone
<i>cléréalisateur</i>
<i>cléemprunt</i>
<i>clécatégorie</i>

catégorie
clé
libellé
<i>cléparente</i>

(My)SQL:

Création de bases de données

- Tables

- `CREATE TABLE nomTable (liste attributs);`
- Liste des attributs:
`nomAttribut typeDonnées (longueur) options`
les divers attributs sont séparés par une virgule
- Attributs: exemple
`titre VARCHAR(50) NOT NULL`

(My)SQL:

Création de bases de données

- Types d'attributs:
 - CHAR() et VARCHAR(): chaînes de caractères
 - INTEGER(): entiers
 - DECIMAL(m,n): valeur numérique avec n décimales
 - DATE, TIMESTAMP: dates
 - TEXT: champ texte de longueur quelconque
 - BLOB: données binaires quelconques (images)

(My)SQL:

Création de bases de données

- Options:
 - PRIMARY KEY: définition de la clé (doit être unique et non nulle)
 - NOT NULL: valeur obligatoire pour l'attribut
 - AUTOINCREMENT: insertion automatique d'une valeur
 - FOREIGN KEY (attribut) REFERENCES table: mise en place de la clé étrangère (depuis la version 4.0 de MySQL)

(My)SQL:

Création de bases de données

- Autres commandes

- `DROP TABLE (IF EXISTS) nomTable`: supprime une table et son contenu
- `ALTER TABLE...` : ajout ou modification d'un attribut
- `CREATE [UNIQUE] INDEX nomIndex on nomTable (attr. 1, attr. 2, ...)`: création d'un index sur les attributs
- Un index permet d'accélérer l'accès aux valeurs des attributs, il peut être utile d'en créer sur les attributs souvent utilisés. Un index est automatiquement créé sur chaque clé primaire

(My)SQL:

Introduction à SQL

- SQL: Structured Query Language:
Language structuré d'interrogation
- Permet une interface standard vers
quasiment tous les SGBD (systèmes de
gestion de bases de données) du
marché
- Norme ANSI (SQL 92 & SQL 99)

(My)SQL:

Introduction à SQL

- Nombreuses différences selon les fabricants (Oracle, IBM, Microsoft, MySQL...)
- Un script SQL Oracle ne fonctionnera pas sous SQL Server ou MySQL
- Il faut connaître les différences pour savoir éviter le SQL propriétaire (si possible)

(My)SQL:

Introduction à SQL

- SQL: la clause SELECT
 - **SELECT** attributs
FROM liste tables
WHERE liste conditions
 - la liste d'attributs peut être remplacée par * si on veut tous les attributs des tables du FROM
 - les attributs doivent impérativement faire partie d'une des tables.

(My)SQL:

Introduction à SQL

- SQL: Conditions
 - Regroupées par des opérateurs logique:
`cond. 1 AND (cond. 2 OR cond. 3)`
 - Sur des attributs:
`prénom = 'QUENTIN'` (attention aux guillemets simples)
 - Permettent les jointures:
`realisateurs.cle = films.cle_etrangere`
 - Les jointures regroupent les données qui ont été séparées dans des tables différentes

(My)SQL:

Introduction à SQL

- SQL: Conditions et chaînes de caractères
 - Il est possible d'utiliser des valeurs indéfinies quand on crée une condition sur des chaînes avec **LIKE**:
 - Tous les prénoms commençant par Q:
`prenom LIKE 'Q%'`
 - Tous les prénoms ne contenant pas T:
`prenom NOT LIKE '%T%'`
- SQL: gestion des valeurs vides (nulles)
 - `prenom IS NOT NULL`

(My)SQL:

Introduction à SQL

- SQL: Clause ORDER BY
 - Permet le classement des résultats selon un attribut
 - `SELECT * FROM films ORDER BY annee;`
 - Inversion de l'ordre: DESC
 - `SELECT * FROM films ORDER BY annee DESC;`

(My)SQL:

Introduction à SQL

- SQL: Clause DISTINCT
 - Supprime l'affichage multiple de valeurs identiques
 - `SELECT DISTINCT annee FROM films ORDER BY annee;`
 - `SELECT annee FROM films ORDER BY annee;`
 - Affiche trois fois 1980 si notre table contient 3 films pour cette année.

(My)SQL:

Introduction à SQL

- SQL: Clause COUNT()
 - Affiche le nombre d'attributs et pas leur valeur
 - `SELECT COUNT(*) FROM films WHERE annee <= 1950;`
 - Affiche le nombre de films datant d'avant 1951

(My)SQL:

Introduction à SQL

- SQL:Agrégation
 - Permet le regroupement des valeurs selon des critères
 - `SELECT annee, COUNT(*) FROM films
GROUP BY annee;`
 - Affiche l'année ainsi que le nombre de films correspondant à cette année
 - Attention, toutes les colonnes du `SELECT` doivent aussi être spécifiées dans le `GROUP BY`

(My)SQL:

Introduction à SQL

- SQL:Agrégation
 - Il y a d'autres fonctions d'agrégation:
 - **AVG**(attribut): moyenne
 - **MIN**(attribut): valeur minimum
 - **MAX**(attribut): valeur maximum
 - **SUM**(attribut): somme

(My)SQL:

Introduction à SQL

- SQL:Agrégation
 - Il est possible d'émettre une restriction sur les groupes par la clause `HAVING condition`
 - `SELECT annee, COUNT(*) FROM films
GROUP BY annee
HAVING COUNT(*) >= 5;`
 - Affiche les années où 5 films au moins ont été réalisés

(My)SQL:

Introduction à SQL

- Insertion de données:
 - `INSERT INTO nomTable VALUES (liste valeurs);`
 - les valeurs doivent correspondre aux attributs (pas possible d'insérer 'QUENTIN' dans un champ DATE)
 - le nombre de valeurs doit correspondre au nombre d'attributs de la table
 - `INSERT INTO nomTable (liste attributs) VALUES (liste valeurs);`
 - permet de n'insérer que dans les attributs listés

(My)SQL:

Introduction à SQL

- **modification des données:**
 - `UPDATE nomTable SET attr1 = valeur1, attr2 = valeur2, ... WHERE conditions;`
 - Attention aux conditions: sans conditions on met à jour toutes les valeurs de la table (rarement utile)

(My)SQL:

Introduction à SQL

- Effacement des données
 - `DELETE FROM nomTable WHERE conditions;`
 - Ici aussi attention aux conditions: pas de conditions et toute la table est vidée...
 - Si possible mettre une condition sur la clé

(My)SQL:

Introduction à SQL

- Spécificités SQL de MySQL
 - Pas de sous-requêtes
 - Pas d'opérateurs ensemblistes
`SELECT ... UNION SELECT...`
 - Pas de gestion des clés étrangères et
 - *Pas de gestion des transactions (MySQL 3.x)*